

รายงาน

เรื่อง การสร้างสมการพหุนามด้วยเมทริกซ์จากภารกิจโดรนส่งเลือดฉุกเฉิน “สายฟ้าแดง”

โดย

นายภูวดล กัสปะ

รหัสนักศึกษา 68050437

เสนอ

รศ. ธีรวัฒน์ ประกอบผล

รายงานนี้เป็นส่วนหนึ่งของรายวิชา พีชคณิตเชิงเส้นสำหรับวิทยาการคอมพิวเตอร์(05506250)

สารบัญ

สารบัญ 2
1. บทนำ 4
2. ที่มาของข้อมูลและเรื่องราว 4
    2.1 เรื่องราวของชุดข้อมูล 4
    2.2 ชุดข้อมูล 5
3. ทฤษฎีที่เกี่ยวข้อง 6
    3.1 พหุนามอินเทอร์โพเลชัน 6
    3.2 เมทริกซ์แวนเดอร์มอนด์ 6
    3.3 การกำจัดเกาส์ 7
4. ขั้นตอนการคำนวณทางคณิตศาสตร์ 7
    4.1 ตั้งระบบสมการ 7
    4.2 กำจัดตัวแปรไปข้างหน้า 8
    4.3 แทนค่าย้อนกลับ 8
    4.4 สมการพหุนามที่ได้ 9
5. การพัฒนาโปรแกรม 9
    5.1 โครงสร้างโปรแกรม 10
    5.2 ลำดับการทำงานเมื่อรันโปรแกรม 10
6. ผังงานและรหัสเทียม 11
    6.1 ผังงาน 11
    6.2 Pseudocode 12
7. ผลการทดลอง 12
    7.1 สัมประสิทธิ์ที่ได้จากโปรแกรม 13
    7.2 ตรวจจุดข้อมูลเดิม 13
    7.3 พยากรณ์ความสูงระหว่างจุด 13
    7.4 กราฟ 14

8. อภิปรายผล 15

9. สรุป 15

10. ภาคผนวก 16

ภาคผนวก ก ไฟล์ชุดข้อมูล 16

ภาคผนวก ข คำสั่งรันโปรแกรม 16

ภาคผนวก ค จุดที่ควรพูดตอนนำเสนอ 16

1. บทนำ

ในชีวิตจริง ความสัมพันธ์ระหว่างตัวแปรสองตัวมักไม่ได้เป็นเส้นตรงเสมอไป เช่น

ความสูงของจรวดเทียบกับเวลา จำนวนผู้ป่วยเทียบกับสัปดาห์ หรือความสูงของโดรนเทียบกับเวลาบิน

ข้อมูลเหล่านี้สามารถประมาณได้ด้วยฟังก์ชันพหุนาม

งานชิ้นนี้เลือกใช้วิธีทางเมทริกซ์ที่เรียนในชั้นเรียน โดยแปลงจุดข้อมูล (x_i, y_i) ให้เป็นระบบสมการเชิงเส้น $Va = y$ แล้วหาสัมประสิทธิ์ของพหุนามด้วยการกำจัดเกาส์

จากนั้นนำขั้นตอนเดียวกันไปเขียนเป็นโปรแกรมภาษา Python เพื่อให้คำนวณซ้ำได้แม่นยำ

และใช้พยากรณ์ค่าในช่วงที่ไม่ได้บันทึกข้อมูล

วัตถุประสงค์ของงานมีดังนี้

1. กำหนดชุดข้อมูลที่มีความสัมพันธ์ระหว่างตัวแปร X และ Y พร้อมเล่าที่มาของข้อมูล
2. สร้างสมการพหุนามด้วยเมทริกซ์แวนเดอร์มอนด์
3. พัฒนาโปรแกรมคำนวณและพยากรณ์ค่า
4. อธิบายการทำงานของโปรแกรมด้วยผังงานและรหัสเทียม
5. เรียบเรียงขั้นตอน ผลลัพธ์ และการตีความลงในรายงานฉบับนี้

2. ที่มาของข้อมูลและเรื่องราว

2.1 เรื่องราวของชุดข้อมูล

คืนหนึ่งช่วงฤดูน้ำหลาก โรงพยาบาลชุมชนบ้านคลองน้ำใสรับผู้ป่วยอุบัติเหตุที่ต้องการเลือดกรุ๊ป O

อย่างเร่งด่วน ถนนสายหลักถูกน้ำท่วม รถพยาบาลจากโรงพยาบาลจังหวัดจึงไปไม่ถึง

หน่วยปฏิบัติการโดรนเวชภัณฑ์ชื่อ **สายฟ้าแดง** จึงถูกสั่งการให้บินส่งถุงเลือดข้ามป่าและแม่น้ำ ระยะทางประมาณ 6 กิโลเมตร ใช้เวลาทั้งภารกิจ 12 นาที โดรนมีเครื่องบันทึกพิกัดและความสูงจาก GPS แต่เพื่อประหยัดพลังงาน ระบบจึงเก็บบันทึกเพียงจุดสำคัญ ไม่ได้เก็บทุกวินาที

เจ้าหน้าที่วางแผนการบินต้องการทราบว่า ในช่วงนาทีที่ไม่มีบันทึก โดรนอยู่สูงเท่าใด โดยเฉพาะนาทีที่ 6 ซึ่งต้องบินข้ามเสาสัญญาณสูงประมาณ 70 เมตร หากประมาณความสูงผิดพลาด โดรนอาจชนสิ่งกีดขวางหรือบินสูงเกินจำเป็นจนสิ้นเปลืองแบตเตอรี่

ดังนั้นในงานนี้ X คือเวลาหลังปล่อยโดรน มีหน่วยเป็นนาที และ Y คือความสูงจากพื้น มีหน่วยเป็นเมตร เป้าหมายคือหาพหุนาม $p(x)$ ที่ผ่านจุดบันทึกทั้งหมด แล้วใช้ $p(x)$ ประมาณความสูง ณ เวลาใดก็ได้ในช่วงภารกิจ

เรื่องนี้สมมติขึ้นเพื่อใช้เป็นบริบทการคำนวณ แต่ลักษณะข้อมูลใกล้เคียงภารกิจโดรนจริง คือเริ่มจากพื้น ไต่ระดับ บินแล่น แล้วลดระดับลงจอด ความสัมพันธ์จึงโค้ง ไม่ใช่เส้นตรง

2.2 ชุดข้อมูล

จุดข้อมูลมาจากบันทึก GPS 4 จุดสำคัญของภารกิจ

จุด	เหตุการณ์	x เวลา (นาที)	y ความสูง (เมตร)
0	ขึ้นจากลานจอดโรงพยาบาลต้นทาง	0	5
1	ข้ามแนวต้นยูคาลิปตัสริมคลอง	4	62
2	บินแล่นคงที่เหนือแม่น้ำ	8	95
3	เข้าใกล้ลานจอดโรงพยาบาลปลายทาง	12	18

เหตุผลที่เลือก 4 จุดมีสองข้อ ข้อแรก จำนวนจุดพอดีกับพหุนามดีกรี 3 ซึ่งโค้งขึ้นแล้วโค้งลงได้สอดคล้องกับการบินขึ้นและลงจอด ข้อที่สอง ระบบสมการที่ได้เป็นสี่เหลี่ยมจัตุรัส แก้ด้วยเมทริกซ์ได้โดยตรง ไม่ต้องใช้วิธีกำลังสองน้อยที่สุด

ข้อมูลเก็บไว้ในไฟล์ `data/drone_mission.csv` เพื่อให้โปรแกรมอ่านค่าเดิมทุกครั้งที่รัน

3. ทฤษฎีที่เกี่ยวข้อง

3.1 พหุนามอินเทอร์โพเลชัน

กำหนดจุด $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ *ที่ค่า* x_i ไม่ซ้ำกัน จะมีพหุนามดีกรีไม่เกิน n เพียงหนึ่งเส้นที่ผ่านทุกจุด คือ

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

เมื่อแทนแต่ละจุดเข้าไป จะได้ระบบสมการ

$$\begin{aligned} a_0 + a_1x_0 + a_2x_0^2 + \cdots + a_nx_0^n &= y_0 \\ a_0 + a_1x_1 + a_2x_1^2 + \cdots + a_nx_1^n &= y_1 \\ &\vdots \\ a_0 + a_1x_n + a_2x_n^2 + \cdots + a_nx_n^n &= y_n \end{aligned}$$

3.2 เมทริกซ์แวนเดอร์มอนด์

ระบบด้านบนเขียนในรูปเมทริกซ์ได้ว่า $Va = y$ โดย

$$V = \left[\begin{array}{cccc|c} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{array}\right]$$

$$a = [a_0\ a_1\ \cdots\ a_n]^T \qquad y = [y_0\ y_1\ \cdots\ y_n]^T$$

เมทริกซ์ V เรียกว่าเมทริกซ์แวนเดอร์มอนด์ (Vandermonde matrix) ถ้า x_i ทุกค่าแตกต่างกันแล้ว det(V) ≠ 0 ดังนั้นมีคำตอบเดียว

ในงานนี้มี 4 จุด จึงใช้พหุนามดีกรี 3 และ V มีขนาด 4 × 4

3.3 การกำจัดเกาส์

แทนที่จะหา V^{-1} โดยตรง ซึ่งคำนวณยากและสะสมความคลาดเคลื่อนง่าย งานนี้แก้ระบบด้วยการกำจัดเกาส์ ซึ่งเป็นวิธีที่เรียนในชั้นเรียน มี 3 ขั้นตอนหลัก

1. สร้างเมทริกซ์เสริม [V | y]
2. ใช้การดำเนินการระหว่างแถว ทำให้ได้รูปสามเหลี่ยมบน
3. แทนค่าย้อนกลับจากแถวล่างขึ้นบน เพื่อหา a_n, a_{n-1}, ..., a_0

โปรแกรมเพิ่มการสลับแถวบางส่วน (partial pivoting) คือเลือกตัวหลักที่มีค่าสัมบูรณ์มากที่สุดในคอลัมน์นั้น เพื่อให้การหารเสถียรขึ้นเมื่อคำนวณด้วยทศนิยม

4. ขั้นตอนการคำนวณทางคณิตศาสตร์

4.1 ตั้งระบบสมการ

แทนจุดทั้งสี่ลงใน $p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$

$$p(0) = a_0 = 5$$
$$p(4) = a_0 + 4a_1 + 16a_2 + 64a_3 = 62$$
$$p(8) = a_0 + 8a_1 + 64a_2 + 512a_3 = 95$$
$$p(12) = a_0 + 12a_1 + 144a_2 + 1728a_3 = 18$$

เขียนเป็นเมทริกซ์เสริม

$$\left[\begin{array}{cccc|c} 1 & 0 & 0 & 0 & 5 \\ 1 & 4 & 16 & 64 & 62 \\ 1 & 8 & 64 & 512 & 95 \\ 1 & 12 & 144 & 1728 & 18 \end{array}\right]$$

4.2 กำจัดตัวแปรไปข้างหน้า

ลบแถวที่ 1 ออกจากแถวที่ 2, 3 และ 4 จะได้

$$\left[\begin{array}{cccc|c} 1 & 0 & 0 & 0 & 5 \\ 0 & 4 & 16 & 64 & 57 \\ 0 & 8 & 64 & 512 & 90 \\ 0 & 12 & 144 & 1728 & 13 \end{array}\right]$$

จากนั้นทำแถวที่ 3 *ให้เป็น แถว*$_3 - 2\cdot$*แถว*$_2$ *และแถวที่* 4 *เป็น แถว*$_4 - 3\cdot$*แถว*$_2$

$$\left[\begin{array}{cccc|c} 1 & 0 & 0 & 0 & 5 \\ 0 & 4 & 16 & 64 & 57 \\ 0 & 0 & 32 & 384 & -24 \\ 0 & 0 & 96 & 1536 & -158 \end{array}\right]$$

สุดท้ายทำแถวที่ 4 *ให้เป็น แถว*$_4 - 3\cdot$*แถว*$_3$

$$\left[\begin{array}{cccc|c} 1 & 0 & 0 & 0 & 5 \\ 0 & 4 & 16 & 64 & 57 \\ 0 & 0 & 32 & 384 & -24 \\ 0 & 0 & 0 & 384 & -86 \end{array}\right]$$

4.3 แทนค่าย้อนกลับ

จากแถวสุดท้าย

$$384a_3 = -86 \implies a_3 = -86/384 = -43/192$$

แทนในแถวที่ 3

$$32a_2 + 384(-43/192) = -24$$

เนื่องจาก 384/192 = 2 จึงได้ $32a_2 - 86 = -24$ ดังนั้น $32a_2 = 62$ และ

$$a_2 = 62/32 = 31/16$$

แทนในแถวที่ 2

$$4a_1 + 16\cdot(31/16) + 64(-43/192) = 57$$

จัดรูปแล้วได้ $4a_1 + 31 - 43/3 = 57$ ดังนั้น $4a_1 = 121/3$ และ

$$a_1 = 121/12$$

แถวแรกให้ $a_0 = 5$ โดยตรง

4.4 สมการพหุนามที่ได้

$$p(x) = 5 + (121/12)x + (31/16)x^2 - (43/192)x^3$$

หรือเขียนทศนิยมประมาณได้ว่า

$$p(x) \approx 5 + 10.083333x + 1.9375x^2 - 0.223958x^3$$

สมการนี้ผ่านจุดข้อมูลทั้งสี่จุดตามทฤษฎีอินเทอร์โพเลชัน

5. การพัฒนาโปรแกรม

โปรแกรมเขียนด้วยภาษา Python เก็บไว้ที่ `src/polynomial_fit.py`

จุดสำคัญคือไม่ได้เรียกฟังก์ชันสำเร็จรูปอย่าง `numpy.polyfit` เพื่อหาสัมประสิทธิ์

แต่สร้างเมทริกซ์และแก้ระบบด้วยตนเองให้ตรงกับที่คำนวณด้วยมือ

5.1 โครงสร้างโปรแกรม

ฟังก์ชัน	หน้าที่
load_dataset	อ่านไฟล์ CSV แล้วตรวจสอบว่ามีจุดพอ และค่า X ไม่ซ้ำ
vandermonde	สร้างเมทริกซ์ V โดยแถวที่ i คือ 1, x_i, x_i^2, ...
gaussian_elimination	แก้ Va = y ด้วยการกำจัดเกาส์และการแทนค่าย้อนกลับ
evaluate_polynomial	คำนวณ p(x) ด้วยวิธีของฮอร์เนอร์
format_equation	แสดงสมการเป็นข้อความอ่านง่าย
verify_fit	แทนจุดเดิมกลับเข้าสมการ เพื่อตรวจว่าคลาดเคลื่อนใกล้ศูนย์
save_plot	วาดกราฟจุดจริงและเส้นโค้งพหุนาม
predict_loop	รับเวลาจากผู้ใช้แล้วพยากรณ์ความสูง

5.2 ลำดับการทำงานเมื่อรันโปรแกรม

1. อ่านชุดข้อมูลจาก data/drone_mission.csv

2. กำหนดดีกรีพหุนามเท่ากับจำนวนจุดลบหนึ่ง

3. สร้างเมทริกซ์แวนเดอร์มอนด์และเวกเตอร์ y

4. แก้ระบบด้วยการกำจัดเกาส์ ได้สัมประสิทธิ์ a_0 ถึง a_3

5. พิมพ์สมการ และตรวจว่า $p(x_i) = y_i$

6. บันทึกกราฟที่ outputs/drone_polynomial.png

7. แสดงตัวอย่างพยากรณ์ที่ x = 6

8. ถ้าไม่ได้ใส่ --no-predict จะรอให้ผู้ใช้ใส่เวลาเพิ่ม

คำสั่งรันโปรแกรม

```
pip install -r requirements.txt
python src/polynomial_fit.py --no-predict
```

ไลบรารีภายนอกใช้เพียง `matplotlib` เพื่อวาดกราฟ

ส่วนการสร้างเมทริกซ์และการกำจัดเกาส์เขียนเองทั้งหมด

6. ผังงานและรหัสเทียม

6.1 ผังงาน

ผังงานการทำงานของโปรแกรม เรียงตามลำดับขั้นตอน

(จุดตัดสินใจแสดงเป็นแขนงย่อยใต้ขั้นตอนที่ตรวจสอบ):

-> เริ่มโปรแกรม

-> อ่านชุดข้อมูล X และ Y จากไฟล์ CSV

-> ตรวจสอบ: จำนวนจุดอย่างน้อย 2 และ X ไม่ซ้ำหรือไม่

  -> แจ้งข้อผิดพลาดแล้วจบ

  -> กำหนดดีกรี n = จำนวนจุด − 1

-> สร้างเมทริกซ์แวนเดอร์มอนด์ V

-> สร้างเวกเตอร์ y

-> รวมเป็นเมทริกซ์เสริม [V | y]

-> กำจัดเกาส์ให้เป็นรูปสามเหลี่ยมบน

-> แทนค่าย้อนกลับหาสัมประสิทธิ์ a

-> แสดงสมการพหุนาม

-> ตรวจจุดข้อมูลเดิม

-> วาดกราฟและบันทึกไฟล์

-> พยากรณ์ความสูงตามเวลาที่ต้องการ

-> จบโปรแกรม

ผังงานนี้ตรงกับฟังก์ชัน run() ในโปรแกรม จุดตัดสินใจอยู่ที่การตรวจชุดข้อมูล เพราะถ้า X ซ้ำ

เมทริกซ์แวนเดอร์มอนด์จะเอกฐานและหาคำตอบเดียวไม่ได้

6.2 Pseudocode

```
อัลกอริทึม สร้างสมการพหุนามด้วยเมทริกซ์
อินพุต: รายการจุด (x[i], y[i]) จำนวน n+1 จุด
เอาต์พุต: สัมประสิทธิ์ a[0], a[1], ..., a[n]


1.  ถ้ามี x ซ้ำกัน แล้วหยุดและแจ้งว่าข้อมูลใช้ไม่ได้
2.  กำหนดดีกรี n = จำนวนจุด - 1
3.  สำหรับ i จาก 0 ถึง n
4.      สำหรับ j จาก 0 ถึง n
5.          V[i][j] <- x[i] ยกกำลัง j
6.  สร้างเมทริกซ์เสริม AUG โดยนำคอลัมน์ y ต่อท้าย V
7.  สำหรับ col จาก 0 ถึง n
8.      หาแถวที่ |AUG[row][col]| มากที่สุด แล้วสลับขึ้นมาเป็นแถวหลัก
9.      ถ้าตัวหลักใกล้ศูนย์ แล้วหยุด เพราะระบบเอกฐาน
10.     สำหรับแถวด้านล่างแต่ละแถว
11.         factor <- AUG[row][col] / ตัวหลัก
12.         ลบ factor คูณแถวหลัก ออกจากแถวนั้น
13. สำหรับ i จาก n ลงมาถึง 0
14.     ผลรวม <- ค่าด้านขวาของแถว i
15.     สำหรับ j จาก i+1 ถึง n
16.         ผลรวม <- ผลรวม - AUG[i][j] * a[j]
17.     a[i] <- ผลรวม / AUG[i][i]
18. คืนค่า a

อัลกอริทึม คำนวณค่าพหุนาม p(x)
1.  value <- 0
2.  สำหรับ coeff ใน a จากตัวท้ายมาตัวแรก
3.      value <- value * x + coeff
4.  คืนค่า value
```

Pseudocode ส่วนแรกคือการกำจัดเกาส์ ส่วนที่สองคือวิธีของฮอร์เนอร์ ซึ่งลดจำนวนครั้งของการยกกำลัง

และลดความคลาดเคลื่อนเมื่อประเมินค่าพหุนาม

7. ผลการทดลอง

7.1 สัมประสิทธิ์ที่ได้จากโปรแกรม

เมื่อรันโปรแกรมกับชุดข้อมูลภารกิจสายฟ้าแดง ได้ค่าดังนี้

สัมประสิทธิ์	ค่าทศนิยม	ค่าเศษส่วน
a_0	5.0000000000	5
a_1	10.0833333333	121/12
a_2	1.9375000000	31/16
a_3	-0.2239583333	-43/192

โปรแกรมและคำนวณด้วยมือได้ค่าเดียวกัน

จึงเชื่อได้ว่าทั้งขั้นตอนทางคณิตศาสตร์และโค้ดทำงานสอดคล้องกัน

7.2 ตรวจจุดข้อมูลเดิม

x	y จากข้อมูล	p(x) จากสมการ	คลาดเคลื่อนสัมบูรณ์
0	5	5.0000	ประมาณ 0
4	62	62.0000	ประมาณ 0
8	95	95.0000	ประมาณ 0
12	18	18.0000	ประมาณ 0

ผลนี้ยืนยันว่าเส้นโค้งอินเทอร์โพเลชันผ่านทุกจุดตามทฤษฎี

7.3 พยากรณ์ความสูงระหว่างจุด

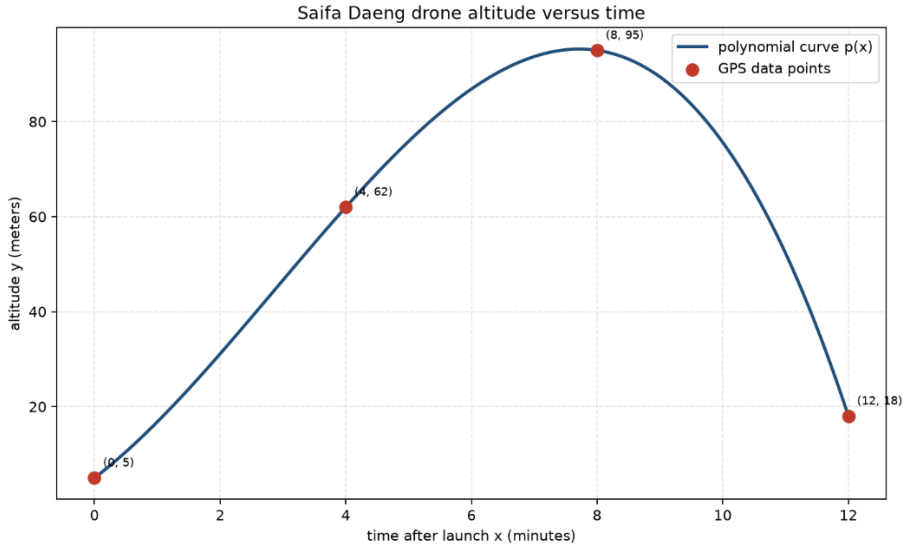
ค่าที่ไม่ได้บันทึกในไฟล์ แต่คำนวณจากสมการได้ เช่น

เวลา x (นาที)	ความสูงประมาณ p(x) (เมตร)	ความหมายในภารกิจ
1	16.80	เพิ่งออกจากลานจอด เริ่มไต่ระดับ
3	46.64	สูงใกล้แนวต้นไม้
6	86.88	ข้ามเสาสัญญาณ 70 เมตรได้ มีระยะเผื่อประมาณ 17 เมตร
9	89.42	เริ่มลดระดับหลังผ่านแม่น้ำ
11	52.27	เข้าสู่ช่วงร่อนลงจอด

คำถามของเจ้าหน้าที่วางแผนจึงตอบได้ว่า ที่นาทีที่ 6 โดรนสูงประมาณ 86.88 เมตร สูงกว่าเสา 70 เมตร จึงผ่านจุดเสี่ยงนี้ได้ตามสมการ

7.4 กราฟ

หลังรันโปรแกรม จะได้ไฟล์ `outputs/drone_polynomial.png` จุดสีแดงคือข้อมูลจริง เส้นสีน้ำเงินคือพหุนาม เส้นโค้งเริ่มที่ 5 เมตร สูงขึ้นจนใกล้ 95 เมตร แล้วลดลงเหลือ 18 เมตร สอดคล้องกับเรื่องการบินขึ้น บินแล่น และลงจอด ป้ายในกราฟใช้ภาษาอังกฤษเพื่อให้ตัวอักษรแสดงครบไม่เพี้ยนตอนส่งออกเป็นรูปภาพ



8. อภิปรายผล

สมการที่ได้ใช้งานได้ดีภายในช่วง 0 ถึง 12 นาที เพราะเป็นช่วงที่มีข้อมูลจริง อย่างไรก็ตาม

พหุนามอินเทอร์โพเลชันมีข้อควรระวังหลายข้อ

ข้อแรก การประมาณนอกช่วงข้อมูลไม่น่าเชื่อถือ เช่น ถ้าแทน x = 20

สมการจะให้ความสูงที่ไม่มีนัยทางภารกิจ เพราะโดรนลงจอดไปแล้วตั้งแต่นาทีที่ 12

ข้อที่สอง ถ้าเพิ่มจุดข้อมูลอีกโดยไม่จำเป็น พหุนามดีกรีสูงอาจแกว่งตัวระหว่างจุด เรียกว่าปรากฏการณ์รุงเง (Runge phenomenon) ดังนั้นจำนวนจุดควรสอดคล้องกับลักษณะโค้งที่ต้องการ

ข้อที่สาม ชุดข้อมูลนี้สมมติขึ้นจากเรื่องราว แม้ขั้นตอนคณิตศาสตร์จะถูกต้อง

แต่ถ้าใช้กับข้อมูลจริงควรมีจุดวัดหนาขึ้น และอาจใช้วิธีกำลังสองน้อยที่สุดเมื่อมีจุดมากกว่าดีกรีที่ต้องการ

ข้อที่สี่ ค่า a_3 เป็นลบ ทำให้พจน์ x^3 ดึงเส้นโค้งลงในช่วงท้าย สอดคล้องกับการลดระดับลงจอด ส่วน $a_0 = 5$ คือความสูงเริ่มต้นบนลานจอดพอดี

แม้โปรแกรมจะใช้ทศนิยม แต่ผลลัพธ์ใกล้เคียงเศษส่วนที่ได้จากมือมาก

เพราะขนาดเมทริกซ์เล็กและการสลับแถวช่วยให้การหารเสถียร

9. สรุป

งานนี้เลือกเรื่องโดรนส่งเลือดฉุกเฉินเป็นบริบทของชุดข้อมูล จากนั้นสร้างพหุนามดีกรี 3

ด้วยเมทริกซ์แวนเดอร์มอนด์และแก้ระบบด้วยการกำจัดเกาส์ ได้สมการ

$$p(x) = 5 + (121/12)x + (31/16)x^2 - (43/192)x^3$$

โปรแกรมภาษา Python ทำขั้นตอนเดียวกันโดยอัตโนมัติ ตรวจได้ว่าเส้นโค้งผ่านจุดเดิมทุกจุด

และใช้พยากรณ์ความสูงระหว่างทางได้ เช่น ที่นาทีที่ 6 ความสูงประมาณ 86.88 เมตร

ซึ่งสูงพอที่จะข้ามเสาสัญญาณ 70 เมตร

การทำงานของโปรแกรมอธิบายด้วยผังงานและรหัสเทียมแล้ว ผู้ใช้สามารถเปลี่ยนไฟล์ CSV

เป็นชุดข้อมูลอื่นได้ โดยยังใช้ทฤษฎีเมทริกซ์ชุดเดิม

10. ภาคผนวก

ภาคผนวก ก ไฟล์ชุดข้อมูล

ไฟล์ `data/drone_mission.csv`

```
time_min,altitude_m,event
0,5,ขึ้นจากลานจอดโรงพยาบาลต้นทาง
4,62,ข้ามแนวต้นยูคาลิปตัสริมคลอง
8,95,บินแล่นคงที่เหนือแม่น้ำ
12,18,เข้าใกล้ลานจอดโรงพยาบาลปลายทาง
```

ภาคผนวก ข คำสั่งรันโปรแกรม

```
python src/polynomial_fit.py --no-predict
python src/polynomial_fit.py
```

คำสั่งแรกใช้ตอนทำรายงานหรือเก็บผลลัพธ์ คำสั่งที่สองใช้ตอนสาธิตการพยากรณ์

ภาคผนวก ค จุดที่ควรพูดตอนนำเสนอ

1. ทำไมใช้เมทริกซ์: จุดข้อมูลกลายเป็นระบบ Va = y

2. ทำไมไม่หาอินเวิร์สโดยตรง: การกำจัดเกาส์สั้นกว่าและเสถียรกว่า

3. ผลที่ตรวจได้ทันที: สมการผ่านจุดเดิม และนาทีที่ 6 สูงกว่าเสา 70 เมตร

4. ข้อจำกัด: อย่าใช้สมการนอกช่วง 0 ถึง 12 นาที